

面向 NPU 核函数的依赖前沿调度与非对称代价溢出规划

高成志 黄骏 叶勤

contact@vennai.org hj992881627@outlook.com yq020319@163.com

摘要

NPU 核函数调度需联合选择微操作 DAG 的合法拓扑序、片上缓冲区的连续地址和容量不足时的溢出计划。本文研究评测器定义的静态非对称代价：有输入备份的缓冲区每次驻留中断计一次传输，生成且无备份的缓冲区计两次，因而 P2 流量可审计地分解为 $Tr = Vol + Dt$ 。我们提出可扩展的有界组合求解器，将新的依赖前沿序与三个既有候选序结合，采用 best-fit 放置和两种驱逐启发式，并直接按 P2 流量或 P3 时间目标选择完整工件。为量化固定拓扑序下的余量，我们建立加权驻留间隙 CP-SAT 松弛；其最优值给出流量下界，若连续地址打包后的工件通过标准评测并达到该下界，即得到固定序流量最优性证书。六个公开 DAG 上，求解器相对官方工件在 P2 中五胜一平（中位降幅 0.866%，最大 9.08%），在 P3 中五快一慢（中位改善 3.77%，唯一回退 4.23%）。我们在两个公开实例的三个固定序上获得证书；其中 Conv_Case0 的依赖前沿序与旧 P1 序的固定序最优流量分别为 57,408 和 81,504 字节，表明该实例中拓扑序可造成 29.6% 的最优流量差距。两个非统一修复个案和内部合成实验仅提供机制、非回归与小规模同序最优性证据，不构成统一生产算法或新的泛化结论。

关键词：NPU 核函数；DAG 调度；片上存储；溢出规划；精确求解

1 引言

NPU 编译器将每个核函数下沉为数千个微操作，这些微操作运行在异构流水线上，并经由多个软件管理的片上缓存交换数据。对此类核函数进行调度，意味着同时确定微操作 DAG 的合法拓扑序、每个驻留缓冲区的连续物理地址，以及容量不足时的溢出计划。合法性约束本身留下巨大的决策自由度：缓冲区在何时分配、使用与释放，既决定必须离开片上的字节总量，也决定哪些数据被选中驱逐；仅凭逻辑活跃峰值无法刻画地址碎片、下次使用距离与非对称溢出代价这三类因素。

基准对有输入备份和生成且无备份的缓冲区分别收取一次与两次传输代价。若总溢出体积为 Vol ，其中生成缓冲体积为 Dt ，则 $Tr = Vol + Dt$ 。因此流量变化必须同时从总溢出量与生成附加量解释；正式定义与紧界见第 3 节，完整证明见补充材料第 2.1 节。

本文将拓扑序构造与物理溢出规划解耦，并以机器可验证的精确规划器为可扩展启发式提供量化标尺（图 1）。生产求解器以**依赖前沿序**避免就绪前驱组饥饿，并在少量结构序、地址/驱逐策略与重载窗口之间按真实 P2/P3 目标选择。固定序研究 oracle 则以加权驻留间隙模型给出流量下界；当所选间隙可连续打包且标准评测值达到下界时，得到固定序流量证书。离线修复与精确规划均作为独立研究工具报告，不进入默认流水线；完整求解过程见补充材料第 3 节。

本文的主要贡献有三：(1) 可扩展的依赖前沿调度器与按真实目标选优的溢出规划组合；(2) 可由连续地址工件升级为固定序流量证书的加权驻留间隙下界；(3) 以 $Tr = Vol + Dt$ 为主线、严格区分生产求解器、修复个案与精确 oracle 的审计式实验体系。在六个公开实例上，生产求解器取得 P2 五胜一平和 P3 时间五胜一负；Conv_Case0 的固定序 oracle 认证 57,408 字节，并揭示相对旧 P1 序的 29.6% 固定序差距。非统一修复与内部合成结果仅作边界明确的附加证据，详见第 5 节及补充材料第 4 节。

代码与复现说明已公开。¹

2 相关工作

联合调度—内存优化。COSMA [8] 已使用 ILP 联合优化算子调度、内存分配与张量替换，故联合优化本身属于既有工作，本文不主张“首次联合”。本文的场景更窄也更细粒度：多缓存微操作 DAG、评测器定义的静态有备份/生成溢出类别、依赖前沿生产启发式，以及将启发式与机器验证最优值相连接的固定序连续地址证书。**寄存器压力与重物化。**指令调度与寄存器分配的联合优化 [4, 10, 11] 同样在容量约束下塑

¹<https://github.com/VennIntelligence/2025A-kernel-sched>

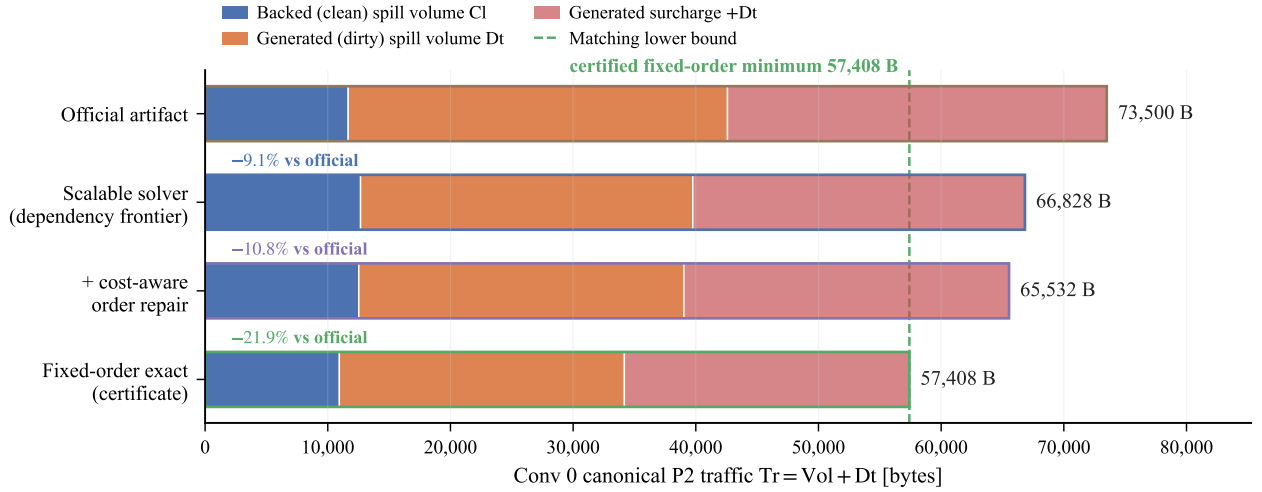


图 1: Conv_Case0 上的精确一启发式证据阶梯, 按恒等式 $Tr = Vol + Dt$ 分解。可扩展生产求解器较官方工件降低 9.1%; 在此基础上, 探索性的代价感知序修复再降 1.9%; 对固定的依赖前沿序, 加权驻留间隙规划器达到自身下界, 从而认证该固定序的流量最优值 57,408 字节。

造活跃区间; Checkmate [6] 离线求解重物化调度, DTR [7] 是其面向动态模型的在线对应。二者均以重计算换取内存, 而本文基准禁止重计算, 要求为静态核函数 DAG 生成显式的溢出/重载操作与具体地址——这正是证书必须包含打包验证的原因。**缓存与替换。**Belady 的固定请求流规则 [1] 与考虑对象大小、代价的缓存研究 [3] 已处理非一致代价, 干净感知替换在编译器内存管理中亦有先例 [5, 9], 故本文不主张一般加权缓存意义上的新颖性。本文的贡献在于其具体组合: 优化器自身可改变的拓扑请求序、多个片上缓存、标准的一次/两次传输溢出代价, 以及“下界—物理打包”桥梁。无溢出编译 [2] 与本文互补: 本文的规划器针对评测器仍要求溢出流量的实例。

3 问题模型

输入为微操作 DAG $G = (\mathcal{V}, \mathcal{E})$ 。操作节点携带流水线、周期数与缓冲区集合; 缓存管理节点为每个缓冲区 b 给出唯一的分配节点 a_b 、释放节点 f_b 、大小 s_b 与缓存类型 $\tau(b)$ 。基础调度是全部原始节点的一个拓扑排列。评测使用五个抽象容量: $L1 = 4096$ 、 $UB = 1024$ 、 $L0A = 256$ 、 $L0B = 256$ 、 $L0C = 512$ 。

评测器对缓冲区作**静态**分类: 若 b 出现在任意 COPY-IN 节点中, 则 $\kappa_b = 1$ (有备份); 否则 $\kappa_b = 2$ (生成且无备份)。一次溢出事件计费 $tr_b = \kappa_b s_b$ 。由于输入不提供逐操作的读写角色, 本文不推断动态 dirty 状态; 该分类是评测器的定义, 而非对硬件一致性协议的断言。令两类累计溢出体积 (有备份/clean、生成/dirty) 为 Cl, Dt , 则

$$Tr = Cl + 2Dt = Vol + Dt, \quad Vol = Cl + Dt, \quad (1)$$

故 $Vol \leq Tr \leq 2Vol$ 。该界说明固定溢出体积时, 类别构成的影响至多为两倍, 但拓扑序和地址规划还可改变 Vol 本身; 紧性证明及适用边界见补充材料第 2.1 节。

物理驻留段必须占据所属缓存内的连续地址区间, 且同时驻留的区间互不重叠。溢出以 SPILL-OUT 与 SPILL-IN 配对中断驻留, 重载后的地址可以不同; 原始使用者与基准的 FREE 事件均为强制驻留事件。P1 仅评价 ALLOC-FREE 的逻辑活跃峰值; P2 输出扩展调度、地址与溢出表, 按键 (Tr, n, T) 排序; P3 按 (T, Tr, n) 排序, 其中 T 为流水线最早完成时间。P1 逻辑峰值不等于溢出后 P2 计划的物理峰值, P3 亦可能有意接受更多流量以换取更短时间, 因此三种视图从不混合比较。所有物理结果均通过标准评测器检查节点覆盖、拓扑合法性、连续地址打包与溢出依赖; 完整评测器契约及 P2-P3 配对权衡见补充材料第 1 节和第 4.4 节。

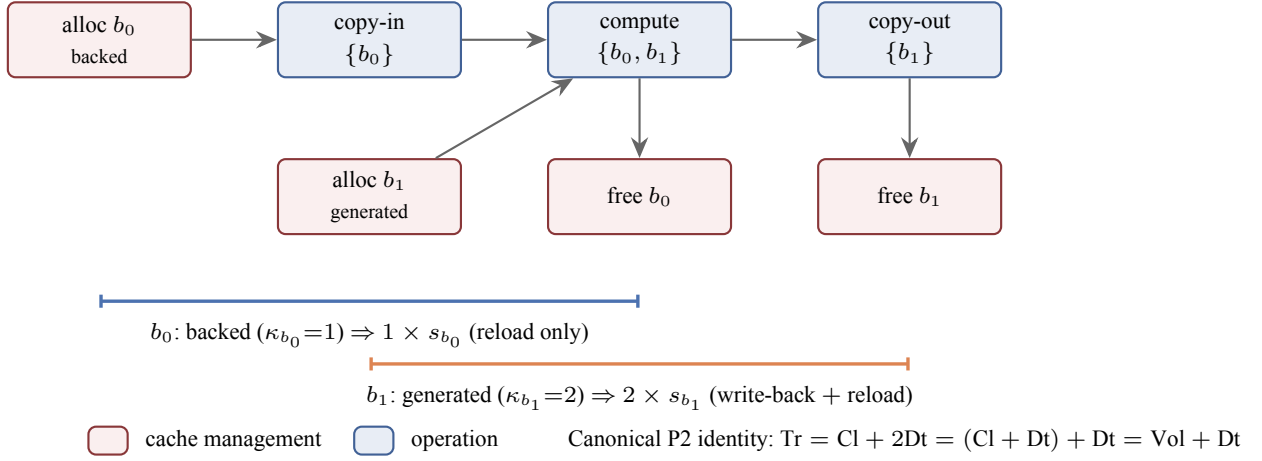


图 2: 微操作 DAG 与基准溢出类别。缓存管理节点界定每个缓冲区的逻辑生命周期: 有输入备份的缓冲区溢出计费一次 (仅重载), 生成且无备份的缓冲区计费两次 (写回加重载)。

4 方法

本文的设计区分三个证据地位不同的角色: **生产求解器**是有界的“候选序 × 在线溢出规划器”组合, 也是唯一进入默认流水线的部分; **代价感知修复**是检验代价语义能否改进拓扑序的离线个案研究; **固定序精确规划器**是提供下界与证书的 CP-SAT 研究 oracle。

4.1 生产求解器

依赖前沿序。 大量同时就绪的 ALLOC 可能使单输入操作流长期饿死一个多输入消费者, 并延长其已就绪操作数的驻留。依赖前沿序优先释放和可执行操作; 仅剩分配节点时, 完成最早可解锁消费者的就绪前驱组 (图 3 与算法 1)。新增前沿序仅使用 DAG 结构与节点编号; 组合内另保留 capfit-id、P1 序与 id-raw 三个既有候选序, 本文不将整个组合重新表述为代价感知或模式无关的调度器。完整平局规则见补充材料第 3.1 节。

地址分配与驱逐策略。 每个基础序使用 best-fit 连续地址分配, 并比较距离/代价评分与自适应碎片策略。两者均为启发式, 最终由完整工件的真实目标择优吸收其局部失误; 精确评分、阈值、搬移边界与平局规则见补充材料第 3.1 节及表 S2。

真实目标组合选择。 P2/P3 分别在四个序、两种策略和 3/5 个重载窗口上形成固定规模组合, 并按 (Tr, n, T) 或 (T, Tr, n) 选择完整工件。不可行成员被跳过, 任何活跃量或溢出面积代理均不参与最终选择; 窗口语义和完整工件生成过程见补充材料第 3.1–3.3 节。

4.2 代价感知修复个案

为探查代价语义进入拓扑序后的潜在余量, 我们离线提出依赖合法的节点或链移动, 重新规划并仅接受 P2 键严格改善的候选。该研究在不同实例上使用不同搜索器和预算, FlashAttention 小预算探针无收益, Matmul 未运行, 因此它不是生产阶段或统一算法。搜索预算、负探针和逐案结果见补充材料第 3.4 节与第 4.5 节。

4.3 固定序加权驻留间隙模型

固定拓扑序后, 每个缓冲区的相邻强制事件形成驻留间隙。保持间隙驻留消耗容量, 中断则支付一次类别加权流量。二元间隙模型在逐缓存、逐切面的容量约束下最小化中断代价; 其解还必须经过连续地址打包、溢出依赖生成与标准评测验证 (图 4)。完整变量定义、容量切面、连续地址反例与模型编码见补充材料第 2.2–2.3 节和第 3.4 节。

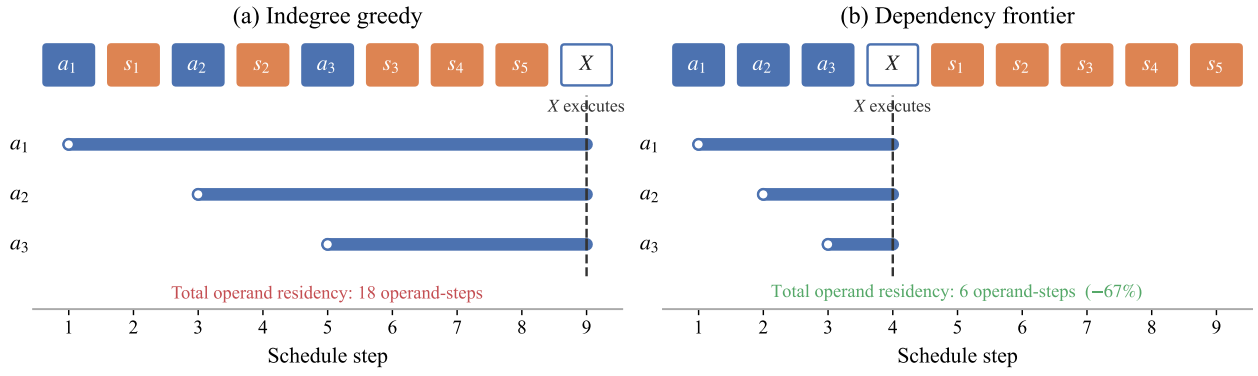


图 3: 前驱组完成机制的示意算例。左: 入度贪心序穿插无关的单输入流, 三操作数消费者 X 迟至第 9 步执行, 累计驻留为 18 操作数步; 右: 依赖前沿序先完成就绪前驱组, 使 X 在第 4 步执行, 累计驻留降至 6 操作数步 (-67%)。

Algorithm 1 Canonical 可扩展求解器

Require: 实例 I , 任务 $p \in \{1, 2, 3\}$

- 1: **if** $p = 1$ **then**
- 2: **return** P1MemoryOrder(I)
- 3: **end if**
- 4: $\mathcal{O} \leftarrow \{\text{Frontier}(I), \text{CapFitID}(I)\}$
- 5: 将 P1Order(I) 与 IDRaw(I) 加入 $\mathcal{O}$
- 6: $\mathcal{R} \leftarrow \{\text{DistCost}, \text{Adaptive25}\}$
- 7: $\mathcal{H} \leftarrow \{0, 5, 40\}$ ($p = 2$), 否则 $\{0, 5, 40, 80, 120\}$
- 8: $A^* \leftarrow \perp$; $k^* \leftarrow (+\infty, +\infty, +\infty)$
- 9: **for all** $(S, r, H) \in \mathcal{O} \times \mathcal{R} \times \mathcal{H}$ **do**
- 10: $A \leftarrow \text{BestFitPlan}(I, S, r, H)$
- 11: **if** A 可行 **then**
- 12: $(\text{Tr}, n, T) \leftarrow \text{Evaluate}(A)$
- 13: $k \leftarrow (\text{Tr}, n, T)$ ($p = 2$), 否则 (T, Tr, n)
- 14: **if** $k < k^*$ **then**
- 15: $(A^*, k^*) \leftarrow (A, k)$
- 16: **end if**
- 17: **end if**
- 18: **end for**
- 19: **return** A^*

▷ 拓扑序、地址偏移与溢出记录

命题 1 (固定序流量下界与证书). 对固定拓扑序, 上述松弛的最优值是该序下一切合法连续地址溢出计划流量的下界; 若某最优间隙选择可完成连续地址打包, 且输出工件通过评测器验证并且流量等于该下界, 则该工件在基准模型下是该固定序的流量最优计划。

证明将任意合法物理计划映射为容量松弛的可行间隙选择; 完整证明以及“容量可行但无法连续打包”的反例见补充材料第 2.2–2.3 节。证书只优化固定序流量, 不优化同流量下的次数或时间, 也不证明跨拓扑序全局最优。

复杂度与部署边界。 在固定候选数下, 可扩展路径为多项式实现, 地址规划含保守的缓冲区二次项; 间隙选择与后备连续打包最坏为指数复杂度。因此 CP-SAT 目前仅作研究 oracle, 未来集成必须提供阈值/超时和经验验证的回退; 详细复杂度见补充材料第 2.5 节。

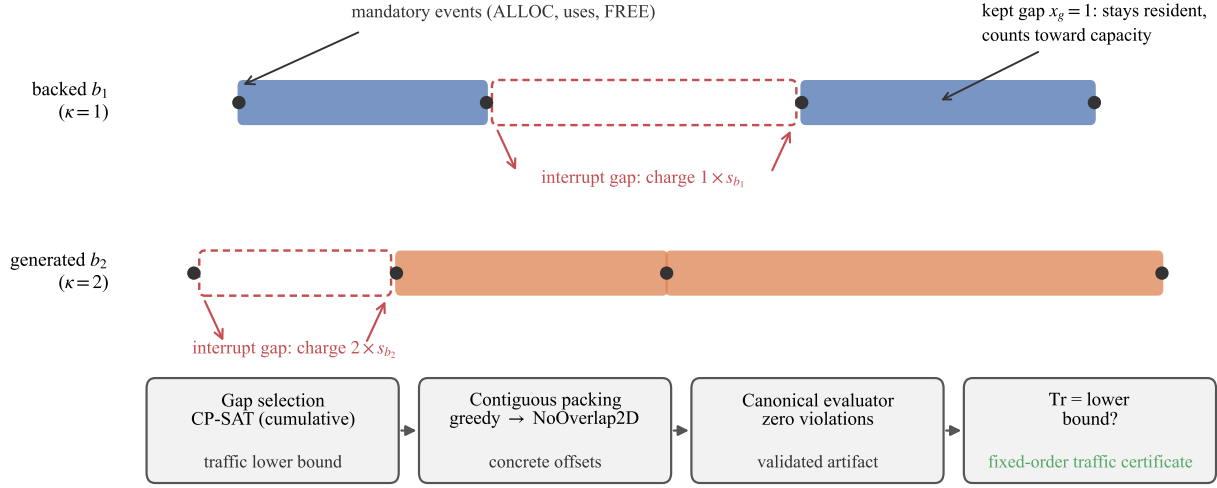


图 4: 加权驻留间隙模型。强制事件将缓冲区生命周期划分为可选间隙：保持驻留占用容量，中断则支付按类别加权的计费 $\kappa_b s_b$ （有备份 $1\times$ 、生成 $2\times$ ）。CP-SAT 松弛给出流量下界；独立的连续地址打包与标准评测验证使达到下界的目标升级为固定序流量证书。

5 实验

六个公开 DAG 来自 2025 年“华为杯”第二十二届中国研究生数学建模竞赛 A 题《通用神经网络处理器下的核内调度问题》发布的数据，覆盖卷积、FlashAttention 与矩阵乘，规模为 1,716–36,086 个节点。全部实验使用第 3 节的标准容量与评测器，所有生产工件均零违规；官方工件仅用于最终比较，任何候选序生成器都不读取官方调度。可扩展求解器是唯一的生产默认；修复个案与精确规划单独报告，以防原型收益或超时被静默当作生产结论。

5.1 公开 P2 与 P3 结果

表 1 与图 5 给出与官方工件的主比较：P2 于五例严格改进、一例持平，中位降幅 0.866%、最大降幅 9.08%；P3 时间于五例改进、一例回退，中位降幅 3.77%，唯一回退是 Conv_1 慢 4.23%。完整 P1/P2/P3 指标、次数、时间平局裁决与验证状态见补充材料表 S5–S7。

受控 $H = 0$ 消融显示组件路径具有描述性非加性，但设计不是完整 factorial；Conv_1 相对实际前代还回退 214 字节，两个 Matmul 实例不变。因此本文只解释完整配置，不报告孤立主效应或协同系数；完整配置表与非加性图见补充材料表 S8 和图 S1。

5.2 $Tr = Vol + Dt$ 机制计账

六例覆盖纯生成、混合与纯有备份三种区间（图 7）。五个严格 P2 胜例均降低 Vol；Conv_1 的 Dt 反而增加 171 字节，Matmul_0 则在 $Dt = 0$ 时获胜。因此不存在统一的类别构成解释；六例逐项计账见补充材料表 S9 和图 S2。

代价感知修复提供“规划器固定、仅换合法拓扑序”的两个非统一个案，分别将 Conv_0/Conv_1 流量降低 1,296/1,794 字节。事后检查发现接受移动伴随有备份 ALLOC/COPY-IN 相关链前移，但 Vol 与 Dt 同时变化，不能据此隔离因果机制。异构预算、FlashAttention 负探针及 Matmul 未运行项见补充材料表 S11。

5.3 精确—启发式证据

Conv_0/前沿序的固定序流量证书为 57,408 字节，较同序可扩展规划低 14.1%；同一 oracle 在旧 P1 序上认证 81,504 字节，隔离出 29.6% 的序差距。FA_0 给出第三份证书并覆盖第二个公开实例。其余实例包含正界限、超时和未运行项（表 2），不能称为流量最优；证书也不优化 P2 次数或时间平局裁决。完整证书链、失败边界与机器工件范围见补充材料第 3.3–3.5 节及表 S12。

表 1: 生产求解器与官方工件的标准 P2/P3 比较。

实例	P2 本文	P2 官方	P3 时间: 本文/官方
Conv_0	66,828	73,500	515,634 / 535,312
Conv_1	72,734	73,240	1,118,687 / 1,073,322
FA_0	3,584	3,692	36,344 / 46,761
FA_1	32,512	32,840	152,364 / 193,059
Matmul_0	34,688	34,944	186,820 / 194,331
Matmul_1	460,800	460,800	1,771,383 / 1,800,218

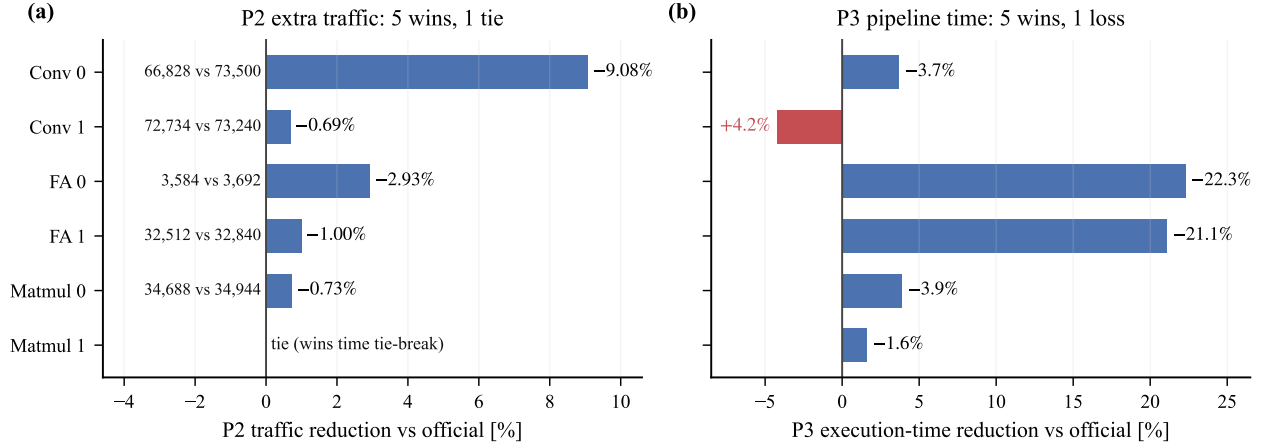


图 5: 六个公开实例上的生产结果。(a) P2 流量相对官方工件的降幅 (标注绝对字节数); (b) P3 执行时间降幅。本图不混入修复个案或固定序 oracle。

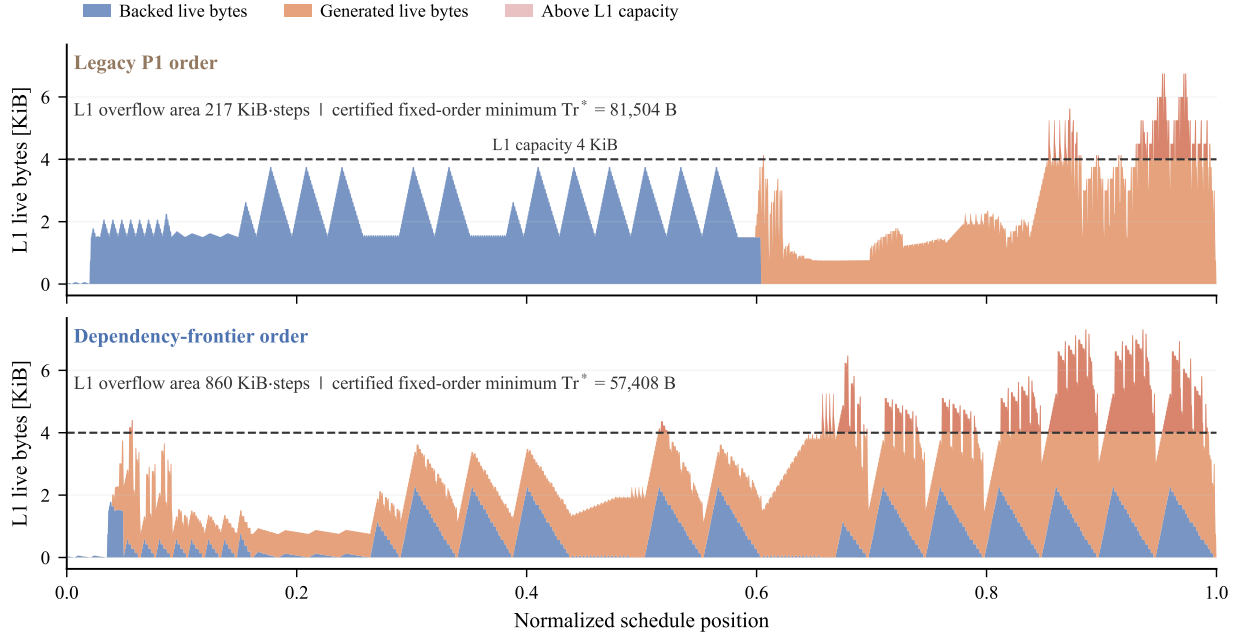


图 6: Conv_Case0 在两个已认证拓扑序下的 L1 逻辑驻留, 并按有备份与生成缓冲分解。依赖前沿序的几何溢出面积更大, 认证的固定序流量最优值却低 29.6% (57,408 对 81,504 字节); 因此在非对称代价下, 几何压力不能可靠地替代真实流量来排序拓扑序。

序空间余量与压力代理。 Conv_0/前沿序的逻辑 L1 溢出面积大于旧 P1 序, 认证流量却低 29.6%。因此活跃量/溢出面积代理不能可靠排序非对称代价下的拓扑序 (图 6)。

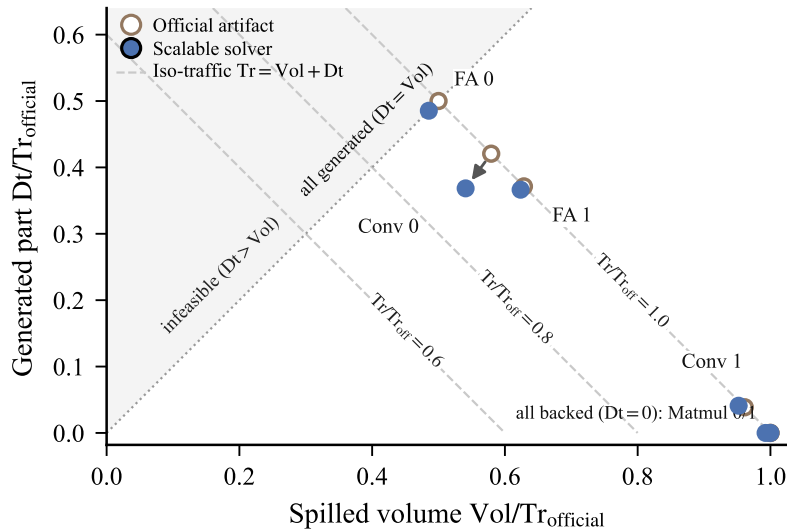


图 7: 六个公开实例在归一化 (Vol, Dt) 平面上的位置。箭头由官方工件指向可扩展求解器；虚线对角线为 $Tr = Vol + Dt$ 的等流量线。实例横跨纯生成 ($Dt = Vol$)、混合与纯有备份 ($Dt = 0$) 三种体制。

表 2: 修复个案与固定序精确侧证据。“-”表示未得到完整结果，而非零流量。

实例	可扩展	修复	固定序	审计状态
Conv_0	66,828	65,532	57,408	流量证书
Conv_1	72,734	70,940	-	打包超时
FA_0	3,584	探针	3,584	流量证书
FA_1	32,512	探针	32,512	可行；下界 31,936
Matmul_0	34,688	-	34,816	可行；下界 29,952
Matmul_1	460,800	-	-	未运行

5.4 稳健性、负结果与限制

容量扫描。在 Conv_0/L1/固定 $H = 0$ 所扫描的全部可行容量点，选定配置从未输给 cp-free-first 序比较者；双方流量均非零时，比值由 $1.35\times$ 增至 $124\times$ 。这是单实例、单缓存的受控边界，不表示绝对差单调或跨算子鲁棒性；完整九点数据、不可行边界与曲线见补充材料表 S13 和图 S4。

内部合成边界。36 例标准复评中，生产组合与其候选子集前代全部持平；cp-free-first 又与被包含的 id-row 序在 36/36 例相同，因此这两组零负主要是组合单调性下的非回归检查。对四组未直接包含的固定实现也观察到零负；8 个 17 节点 oracle 均达到自身所选序的固定序流量最优。该证据支持非回归和小规模同序吻合，不证明相对前代的新泛化。36+8 逐实例结果与 252 次运行时验证见补充材料表 S14–S17；工件位置和复现命令由仓库运行说明维护。

范围威胁。公开评测仅含六个 DAG，且精确研究不完整；基准将出现在 COPY-IN 中的缓冲区静态标记为有备份、要求 FREE 时驻留、不暴露读写角色，因而排除任何关于动态 dirty 状态演化的定理；P1 报告逻辑 ALLOC-FREE 足迹而非物理 P2 驻留；CP-SAT 可能超时，可扩展驱逐策略仍是启发式。本文全部结论限于所述评测器与工件契约。

6 结论

本文将 NPU 核函数内存规划重构为两个可分离、可审计的问题：哪个合法拓扑暴露出可处理的驻留前沿，以及容量受限时应保持哪些有备份或生成的驻留间隙。生产层面的答案是围绕依赖前沿序、按真实目标选择的有界组合；固定序层面的研究答案是加权间隙下界与连续地址流量证书。恒等式 $Tr = Vol + Dt$ 使每次 P2 变化都可分解审计。

证据有效但边界明确：公开实例上 P2 于五例改进、一例持平，P3 时间于五例改进、一例回退；两个非统一修复个案改进两个卷积实例；经标准验证的合成套件复评确立非回归与小规模同序流量最优，而非相对前代的新泛化。因此本文的结论不是普适的 clean/dirty 调度定律，而是一座边界明确的精确—启发式桥梁，以及在所研究核函数族上经验证的改进。后续工作应在超时与验证回退的保护下集成精确求解、设计更廉价且统一的代价感知序修复，并在读写语义比当前静态标签更丰富的硬件轨迹上评估本方法。

参考文献

- [1] L. A. Belady. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, 5(2): 78–101, 1966.
- [2] Prasanth Chatarasi, Alex Gatea, Wei Wang, Chris Bowler, Shubham Jain, Masoud Ataei Jaliseh, Nicole Khoun, Alberto Mannari, Bardia Mahjour, Viji Srinivasan, and Swagath Venkataramani. Enabling spill-free compilation via affine-based live range reduction optimization. In *Proceedings of the IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 1–13, 2026.
- [3] Christian Coester, Roie Levin, Joseph (Seffi) Naor, and Ohad Talmon. Competitive algorithms for block-aware caching. In *Proceedings of the 34th ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*, 2022.
- [4] James R. Goodman and Wei-Chung Hsu. Code scheduling and register allocation in large basic blocks. In *Proceedings of the 2nd International Conference on Supercomputing (ICS)*, pages 442–452, 1988.
- [5] Jia Guo, Maria Jesus Garzaran, and David Padua. The power of Belady’s algorithm in register allocation for long basic blocks. In *Languages and Compilers for Parallel Computing (LCPC)*, volume 2958 of *Lecture Notes in Computer Science*, pages 374–389. Springer, 2004. doi: 10.1007/978-3-540-24644-2_24.
- [6] Paras Jain, Ajay N. Jain, Aniruddha Nrusimha, Amir Gholami, Pieter Abbeel, Kurt Keutzer, Ion Stoica, and Joseph E. Gonzalez. Checkmate: Breaking the memory wall with optimal tensor rematerialization. In *Proceedings of Machine Learning and Systems (MLSys)*, 2020.
- [7] Marisa Kirisame, Steven Lyubomirsky, Altan Haan, Jennifer Brennan, Mike He, Jared Roesch, Tianqi Chen, and Zachary Tatlock. Dynamic tensor rematerialization. *arXiv preprint*, abs/2006.09616, 2020.
- [8] Yi Li, Aarti Gupta, and Sharad Malik. Combined scheduling, memory allocation and tensor replacement for minimizing off-chip data accesses of DNN accelerators. *arXiv preprint*, abs/2311.18246, 2023.
- [9] Vincenzo Liberatore, Martin Farach-Colton, and Ulrich Kremer. Evaluation of algorithms for local register allocation. In *Proceedings of the 8th International Conference on Compiler Construction (CC)*, volume 1575 of *Lecture Notes in Computer Science*, pages 137–153. Springer, 1999. doi: 10.1007/978-3-540-49051-7_10.
- [10] Ghassan Shobaki, Saeed Shawabkeh, and Najm Eldeen Abu-Rmaileh. Preallocation instruction scheduling with register pressure minimization using a combinatorial optimization approach. *ACM Transactions on Architecture and Code Optimization*, 10(4), 2013.
- [11] Ghassan Shobaki, Ahmad Al Qutami, and Najm Eldeen Abu-Rmaileh. Register-pressure-aware instruction scheduling using ant colony optimization. *ACM Transactions on Architecture and Code Optimization*, 19(4), 2022.